

Must-fix decisions before build

These corrections supersede any older wording in the detailed PRD. They are mandatory for the SIH build and must be reflected in code, environment variables, testing, and the demo runbook.

Priority	Decision	Implementation requirement
MUST-FIX 1	AYUSH differentiation	Add a dedicated Dashavidha Pariksha intake branch covering Prakriti, Vikriti, Agni, Koshtha, and Sattva.
MUST-FIX 2	Voice latency risk	Use hosted Groq Whisper API as the primary transcription path. Keep browser Web Speech API as a silent fallback. Do not add self-hosted Docker Whisper for the demo.
MUST-FIX 3	Demo deployment risk	Use hosted staging as the primary demo environment: Vercel frontend, Railway API/worker, and managed PostgreSQL. Local Docker Compose remains a development fallback only.

1. AYUSH intake branch

The intake is not generic/allopathic only. Patients may choose the AYUSH pathway when the clinic enables it. The pathway collects Dashavidha Pariksha inputs in a culturally respectful, clinician-reviewable format. It is structured intake, not autonomous diagnosis.

Section	Minimum capture	Output label
Prakriti	patient-reported constitution observations and optional clinician notes	AYUSH - Prakriti
Vikriti	current imbalance / symptom pattern as reported	AYUSH - Vikriti
Agni	appetite, digestion, regularity, and patient language	AYUSH - Agni
Koshtha	bowel pattern and relevant patient-reported context	AYUSH - Koshtha
Sattva	wellbeing, sleep, stress, and mental state prompts	AYUSH - Sattva

2. Voice and deployment architecture

Area	Primary path	Fallback / degraded mode	Acceptance test
Speech	Browser records audio -> Railway worker -> hosted Groq Whisper API	Browser Web Speech API is silent fallback; manual typing always available	Provider timeout does not block intake.
Frontend	Vercel staging deployment	Local Next.js only for development	Demo URL works before event day.
API and worker	Railway service with health/readiness checks	Local process for development	Worker retries are observable.
Database	Managed PostgreSQL	Local PostgreSQL via Docker Compose	Migration and seed run against staging.
Storage	Hosted private object storage	Local S3-compatible storage	Signed access and retention tested.

3. Required environment variables

Each teammate creates a local .env from .env.example. Real secrets stay in the hosting provider secret store or local uncommitted .env. Never commit keys.

Variable	Local/mock value	Staging value
AI_PROVIDER_MODE	mock	groq
GROQ_API_KEY	blank	Railway secret
WHISPER_PROVIDER_MODE	mock	groq-hosted
WEB_SPEECH_FALLBACK	true	true
AYUSH_INTAKE_ENABLED	true	true
DATABASE_URL	local Postgres	Railway managed Postgres URL
NEXT_PUBLIC_API_URL	http://localhost:8000	Railway API URL
JWT_SECRET	random local secret	Railway secret

4. Module ownership changes

Module	Required change
Backend lead	Add typed settings, provider adapter interface, staging deployment files, CORS for Vercel, readiness checks, and shared AYUSH enums.
Auth	Store selected intake pathway and AYUSH consent language; preserve patient privacy and role scope.
AI Summary	Summarize AYUSH fields with explicit provenance; never merge model inference into clinician-confirmed facts.
Whisper and OCR	Implement Groq Whisper adapter, browser fallback contract, retry/status states, and hosted storage configuration.
Doctor Queue	Show pathway badge in queue and doctor workspace; allow doctor to filter/review AYUSH visits without changing FIFO rules.

5. Final Antigravity instruction

Read BASE_PRD_MediKiosk.pdf first, then the assigned module PRD. Apply the three mandatory final corrections: implement the Dashavidha Pariksha AYUSH intake branch, use hosted Groq Whisper as primary with browser Web Speech API as silent fallback, and target Vercel + Railway + managed PostgreSQL staging for the demo. Do not implement self-hosted Docker Whisper as the primary demo path. Keep API keys in environment variables or hosting secrets, never source control. Work only in the assigned branch and owned files, add tests, and document the staging integration.

These changes are mandatory acceptance criteria for every module, even when the module only consumes the shared contracts.

MediKiosk

AI-enabled clinic operations for Bharat

Backend Lead PRD

Backend lead | Module implementation contract

v1.0 | 28 Aug 2026

Implementation-ready product requirements package

Document control

Field	Value
Document	Backend Lead PRD
Owner	Backend lead
Status	Build-ready baseline
Audience	SIH team, reviewers, Antigravity agent
Expected length	20+ pages
Purpose	Own the shared platform foundation and integration seams.

How to use this document

Read the BASE_PRD first, then the relevant module PRD. Treat API contracts, ownership boundaries, validation rules, and acceptance criteria as the source of truth. If implementation pressure requires a change, record it in the decision log and keep branch ownership intact.

1. Module charter and boundaries

Own the shared platform foundation and integration seams. This module depends on the BASE_PRD for shared terminology, API envelope, privacy principles, and branch rules. Its job is to implement a narrow, testable capability with explicit seams for neighboring modules.

In scope	Out of scope	Integration dependency
FastAPI architecture and related business rules	Unrelated UI redesign, billing, clinical diagnosis	Shared settings, DB session, auth dependency
SQLAlchemy models with typed contracts	Changing other module tables without agreement	Visit and patient identifiers
JWT and middleware with observable statuses	Provider-specific logic in route handlers	Repository/service conventions
Migrations and Swagger plus failure paths	Production infrastructure beyond documented adapter	Audit and request ID helpers

2. User stories

- As a developer, I can start the API and database from a clean checkout.
- As a module owner, I can add a route without duplicating auth, settings, or error handling.
- As an operator, I can distinguish liveness, readiness, provider failure, and database failure.
- As a reviewer, I can inspect migrations, OpenAPI, tests, and integration notes.

3. Owned files and folder structure

Owned paths

```
app/core/ settings.py, security.py, logging.py
app/db/ session.py, models/, migrations/
app/api/ router.py, dependencies.py
app/schemas/ common.py
app/tests/ test_health.py, test_contracts.py
```

Do not modify without agreement: shared model names, public envelope, migration ordering, global settings keys, or another branch's route/service files. Add an integration note when a shared contract must change.

4. Sequence design

Step	Contract
1	Client -> router: request
2	router -> dependency: auth/request ID
3	router -> service: validated command
4	service -> repository: transaction
5	service -> adapter: external call
6	service -> router: typed response

5. API contracts

Method	Path	Auth	Request / response example
GET	/health	none	{ "success": true, "data": { "status": "ok" } }
GET	/ready	none	{ "success": true, "data": { "database": "ok", "storage": "ok" } }
GET	/openapi.json	none	OpenAPI document served by FastAPI

Contract rules

Every response uses the shared success/error envelope. Request examples are illustrative; Pydantic schemas are authoritative. Reject unknown enum values, return 409 for idempotency or state conflicts, 404 for inaccessible resources, and 422 for validation failures. Never return password hashes, provider credentials, raw access tokens, or unredacted sensitive logs.

6. Data interactions and invariants

Entity	Interaction	Invariant
users / patients / visits	Shared SQLAlchemy models and relationships	Foreign keys, UTC timestamps, migrations, transaction boundary
audit_events	Append-only event writer	Actor, action, entity, request ID always present
settings	Environment and typed config	Fail fast on missing required production secrets

7. Low-fidelity testing UI

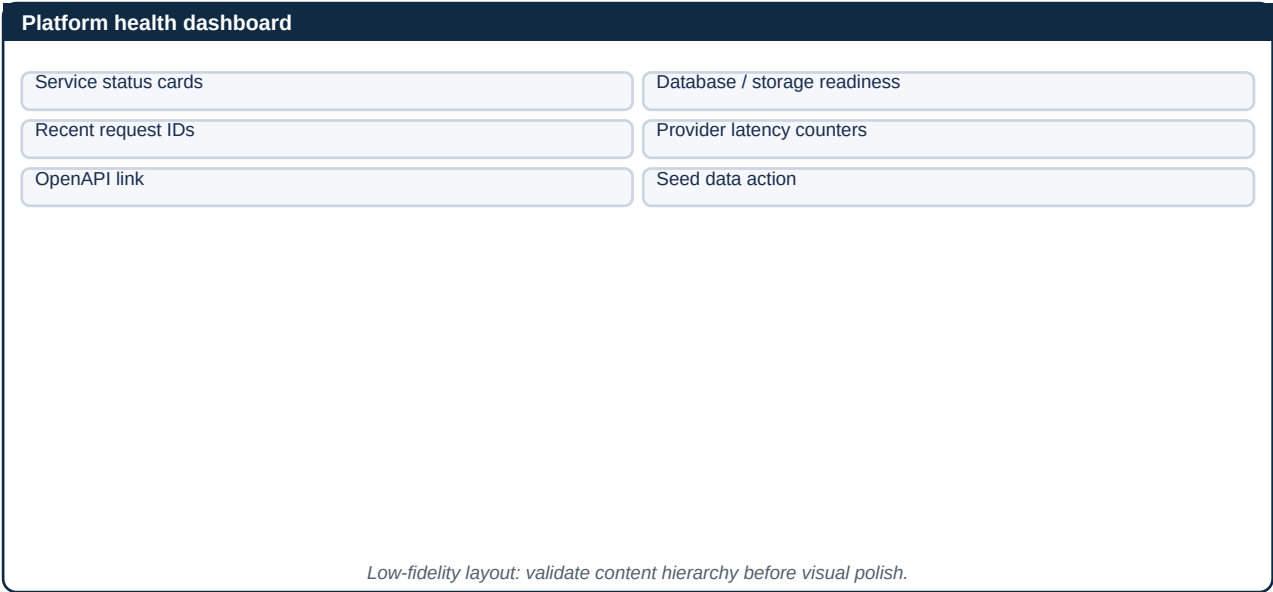


Figure. Minimal test UI exists to prove API behavior, not to become the final product interface.

8. Test plan and acceptance criteria

Acceptance ID	Given / when / then
BACKEND-01	Clean startup passes with documented environment.
BACKEND-02	Health is 200 only when process is alive; readiness reflects dependencies.
BACKEND-03	OpenAPI contains tags, auth schemes, request bodies, and error examples.
BACKEND-04	Database session rolls back on service failure and tests run isolated.
BACKEND-05	Request IDs appear in response metadata and structured logs.

9. Integration and merge checklist

Before PR	Required evidence
Contracts	Swagger diff or endpoint examples updated.
Database	Migration applies forward on clean DB and rollback strategy noted.
Tests	Unit, API, failure-path, and authorization tests pass.
Demo	Minimal testing page demonstrates happy path and one failure path.
Docs	README section, env vars, seed instructions, and integration notes included.
Review	No unrelated files changed; branch ownership respected; screenshots attached.

10. Final Antigravity prompt

Read `BASE_PRD_MediKiosk.pdf`, then read Backend Lead PRD. Work only in the assigned branch and owned paths. Implement the API contracts, database interactions, service boundaries, validation rules, tests, and minimal Next.js testing UI described here. Follow FastAPI + SQLAlchemy + Pydantic conventions, use the shared response envelope, preserve consent and audit requirements, and do not redesign or rewrite other modules. Finish with migrations, Swagger examples, failure handling, and a short integration note for the backend lead.

11. Settings matrix

Environment keys are typed, validated, and documented. Required keys: `DATABASE_URL`, `JWT_SECRET`, `JWT_EXPIRE_MINUTES`, `STORAGE_ENDPOINT`, `STORAGE_BUCKET`, `AI_PROVIDER_MODE`, `LOG_LEVEL`. Local defaults may use mocks; production must fail fast when secrets are absent.

Verification	Expected
Happy path	Status and response match contract.
Failure path	Stable error code and no partial mutation.
Authorization	Wrong role or scope is rejected.
Observability	Request ID and audit behavior are visible.

12. SQLAlchemy modeling rules

Use UUID primary keys, timezone-aware datetimes, explicit relationship loading, named constraints, and repository queries that encode clinic scope. Keep provider payloads in JSON columns only where the schema is versioned and validated.

Verification	Expected
Happy path	Status and response match contract.
Failure path	Stable error code and no partial mutation.
Authorization	Wrong role or scope is rejected.
Observability	Request ID and audit behavior are visible.

13. Middleware contract

Install request ID, structured error handling, CORS allowlist, auth parsing, and timing middleware. Middleware must not make database calls except the explicit auth dependency.

Verification	Expected
Happy path	Status and response match contract.
Failure path	Stable error code and no partial mutation.
Authorization	Wrong role or scope is rejected.
Observability	Request ID and audit behavior are visible.

14. Migration playbook

Create migration, apply on empty database, apply on seeded database, verify indexes, then run API tests. Never edit an applied migration; add a new revision.

Verification	Expected
Happy path	Status and response match contract.
Failure path	Stable error code and no partial mutation.
Authorization	Wrong role or scope is rejected.
Observability	Request ID and audit behavior are visible.

15. Integration test dashboard

Show readiness, route inventory, DB migration head, seed status, and recent error counts. This dashboard is a test harness, not a clinical admin screen.

Verification	Expected
Happy path	Status and response match contract.
Failure path	Stable error code and no partial mutation.
Authorization	Wrong role or scope is rejected.
Observability	Request ID and audit behavior are visible.

16. Backend acceptance walkthrough

Start services, open Swagger, create a seeded request, inspect request ID, stop storage, verify readiness turns degraded, restore dependency, and rerun the request safely.

Verification	Expected
Happy path	Status and response match contract.
Failure path	Stable error code and no partial mutation.
Authorization	Wrong role or scope is rejected.
Observability	Request ID and audit behavior are visible.

17. Backend handoff notes

Document every shared model, dependency, error code, migration, and unresolved assumption for the module branches.

Verification	Expected
Happy path	Status and response match contract.
Failure path	Stable error code and no partial mutation.
Authorization	Wrong role or scope is rejected.
Observability	Request ID and audit behavior are visible.

18. Backend review rubric

Prefer small service functions, explicit transactions, typed boundaries, deterministic tests, and readable failure behavior.

Verification	Expected
Happy path	Status and response match contract.
Failure path	Stable error code and no partial mutation.
Authorization	Wrong role or scope is rejected.
Observability	Request ID and audit behavior are visible.

19. Backend risk register

Primary risks are hidden coupling, migration drift, inconsistent errors, and secrets in local config. Each is mitigated by contract tests, migration checks, shared envelope, and .env.example.

Verification	Expected
Happy path	Status and response match contract.
Failure path	Stable error code and no partial mutation.
Authorization	Wrong role or scope is rejected.
Observability	Request ID and audit behavior are visible.

20. Backend completion statement

The module is complete when a clean checkout starts, all routes are discoverable, readiness is meaningful, migrations are repeatable, and other branches can integrate without copying platform code.

Verification	Expected
Happy path	Status and response match contract.
Failure path	Stable error code and no partial mutation.
Authorization	Wrong role or scope is rejected.
Observability	Request ID and audit behavior are visible.

21. Transaction boundaries

Define one transaction per business command. Repository reads may be composed, but writes commit only in the service boundary. External provider calls must not hold a database lock.

Verification	Expected
Clean checkout	Setup is reproducible.
Contract	Swagger and response examples agree.
Failure drill	Dependency outage is safe and visible.

22. API compatibility checklist

When a schema changes, add an example, update OpenAPI, add a backward-compatible parser where needed, and notify every module owner before merge.

Verification	Expected
Clean checkout	Setup is reproducible.
Contract	Swagger and response examples agree.
Failure drill	Dependency outage is safe and visible.

23. Local development runbook

Start PostgreSQL and object storage, apply migrations, seed clinic and users, run tests, open Swagger, and confirm readiness before handing the branch to contributors.

Verification	Expected
Clean checkout	Setup is reproducible.
Contract	Swagger and response examples agree.
Failure drill	Dependency outage is safe and visible.

24. Platform failure drills

Stop the database, storage, and AI mock one at a time. Confirm readiness, error envelopes, retry behavior, and recovery without corrupting state.

Verification	Expected
Clean checkout	Setup is reproducible.
Contract	Swagger and response examples agree.
Failure drill	Dependency outage is safe and visible.

25. Backend final sign-off

The shared foundation is accepted when module branches can depend on stable settings, sessions, models, audit, errors, and OpenAPI without copying implementation code.

Verification	Expected
Clean checkout	Setup is reproducible.
Contract	Swagger and response examples agree.
Failure drill	Dependency outage is safe and visible.

Environment and API key setup

Each teammate creates a local environment file after checking out their branch. Never paste real keys into a PRD, source file, GitHub issue, screenshot, or committed .env file. Commit only .env.example with placeholder values.

Setup steps

- From the repository root, copy .env.example to .env.
- Fill the values locally. Use the team's approved provider accounts or local mock mode for development.
- Start the database, object storage, and API using the documented Docker Compose command.
- Open /health and /ready, then open /docs to confirm the API is configured.
- If a key is rotated, update only local secrets or the team secret store; never amend old commits to hide a leaked key.
- Before pushing, verify .env is listed in .gitignore and run the repository secret scan.

Recommended .env.example

Safe placeholder file

```
APP_ENV=local
DATABASE_URL=postgresql+psycopg://medikiosk:medikiosk@localhost:5432/medikiosk
JWT_SECRET=replace-with-a-long-random-local-secret
JWT_EXPIRE_MINUTES=30
STORAGE_ENDPOINT=http://localhost:9000
STORAGE_ACCESS_KEY=local-access-key
STORAGE_SECRET_KEY=local-secret-key
STORAGE_BUCKET=medikiosk-private
AI_PROVIDER_MODE=mock
OPENAI_API_KEY=
WHISPER_PROVIDER_MODE=mock
OCR_PROVIDER_MODE=mock
CORS_ORIGINS=http://localhost:3000
LOG_LEVEL=INFO
```

Which values are real secrets?

Variable	Purpose	Who provides it	Safe local default
OPENAI_API_KEY	GPT summary provider	Backend lead / approved account	Blank when AI_PROVIDER_MODE=mock
STORAGE_SECRET_KEY	Private object storage	Operator	Local MinIO secret
JWT_SECRET	Signs access tokens	Backend lead	Generated random local value
DATABASE_URL	Database credentials	Backend lead	Local Docker database
OCR / Whisper key	Speech or OCR provider	Media module owner	Blank when provider mode is mock

Branch rule: every teammate uses their own .env on their own branch checkout. The same variable names must work across branches. If a new variable is needed, add its placeholder to .env.example and tell the backend lead before merging.

Shared implementation checklist

Check	Definition of done
Local setup	README, .env.example, migrations, seed data, and one-command startup documented.
API quality	OpenAPI tags, consistent envelope, validation errors, auth dependencies, and correlation IDs.
Data quality	UTC timestamps, foreign keys, indexes for queue/history lookups, and soft-delete policy.
Security	Secrets outside source, password hashing, JWT expiry, upload limits, and audit events.
Demo readiness	Seeded clinic, test users, sample transcript/document, and a happy-path script.

Decision log template

Date	Decision	Reason	Owner
YYYY-MM-DD	Short decision statement	Tradeoff or constraint	Name / role
YYYY-MM-DD	Short decision statement	Tradeoff or constraint	Name / role